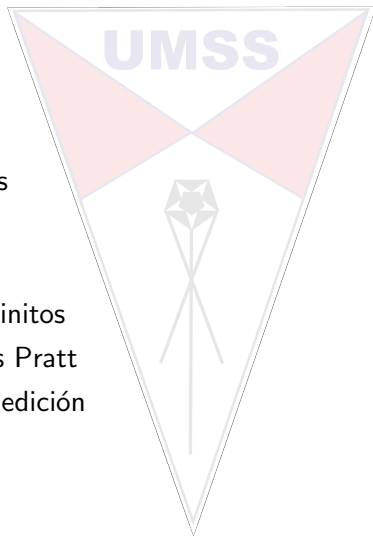
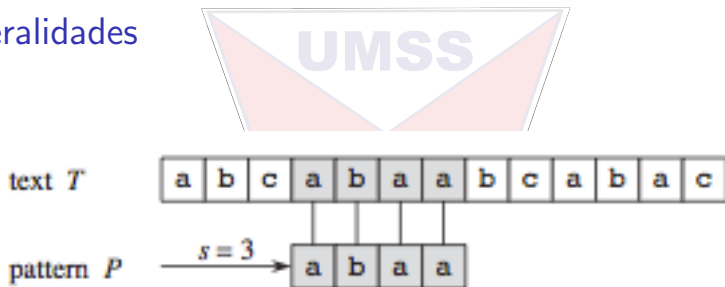




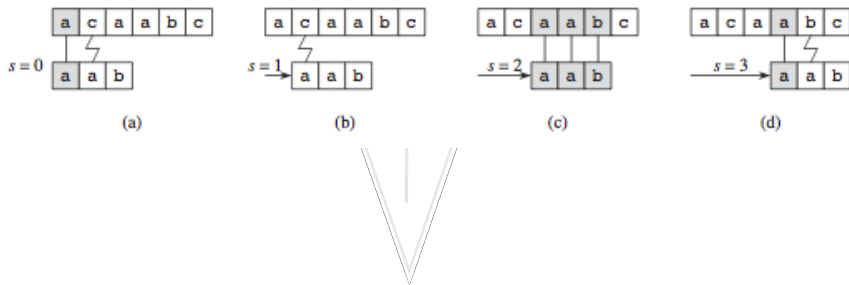


Generalidades



- ▶ P, T cadenas texto y patron de búsqueda
- ▶ Σ alfabeto sobre el cual las cadenas se construyen, T y P
- ▶ $|\Sigma|$ tamaño del alfabeto

Se busca recorriendo de uno en uno los caracteres dentro el texto:



te

```

1   $n = T.length$ 
2   $m = P.length$ 
3  for  $s = 0$  to  $n - m$ 
4      if  $P[1..m] == T[s + 1..s + m]$ 
5          print "Pattern occurs with shift"  $s$ 

```



numerales para el texto tomado

Los equivalentes numerales para el texto tomados de $|P|$ caracteres son:

2 3 5 9 0 2 3 1 4 1 5 2 6 7 3 9 9 2 1

8 9



Los equivalentes numerales para el texto tomados de $|P|$ caracteres son:

2 3 5 9 0 2 3 1 4 1 5 2 6 7 3 9 9 2 1

8 9 3

y así...

numerales para el texto tomado

Los equivalentes numerales para el texto tomados de $|P|$ caracteres son:

2 3 5 9 0 2 3 1 4 1 5 2 6 7 3 9 9 2 1



8 9 3 11 0 1 7 8 4 5 10 11 7 9 11

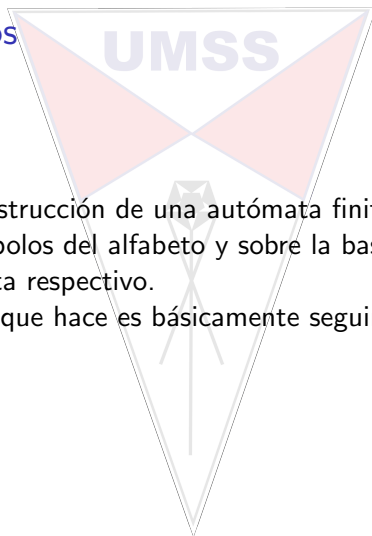
Este proceso tiene complejidad $O(m \cdot n)$


```

1   $n = T.length$ 
2   $m = P.length$ 
3   $h = d^{m-1} \bmod q$ 
4   $p = 0$ 
5   $t_0 = 0$ 
6  for  $i = 1$  to  $m$  // preprocessing
7       $p = (dp + P[i]) \bmod q$ 
8       $t_0 = (dt_0 + T[i]) \bmod q$ 
9  for  $s = 0$  to  $n - m$  // matching
10     if  $p == t_s$ 
11         if  $P[1..m] == T[s + 1..s + m]$ 
12             print "Pattern occurs with shift"  $s$ 
13     if  $s < n - m$ 
14          $t_{s+1} = (d(t_s - T[s + 1])h) + T[s + m + 1]) \bmod q$ 

```

En el peor caso $O(n*m)$ dependerá de la función que se use para que su tendencia sea menor.



S

UMSS

Construcción de una autómatata finit
bolos del alfabeto y sobre la base
a respectivo.

que hace es básicamente seguir

Se basa en la construcción de una autómata finito determinístico que tome los símbolos del alfabeto y sobre la base de éstos se genere el autómata respectivo.

Este algoritmo lo que hace es básicamente seguir un flujo de emparejamiento.



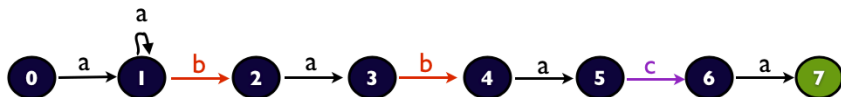
5 - Construcción de autó

→ a
→ b
→ c



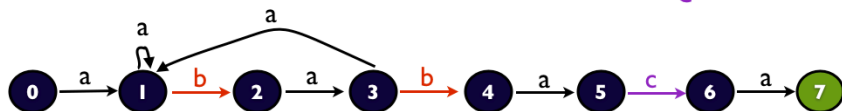
5 - Construcción de autó

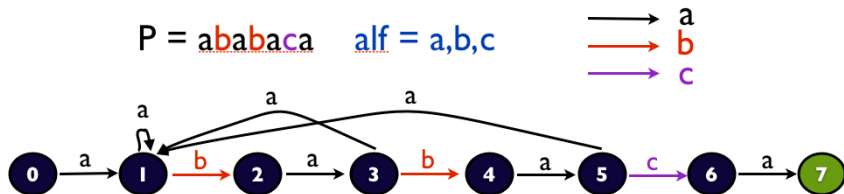
→ a
→ b
→ c



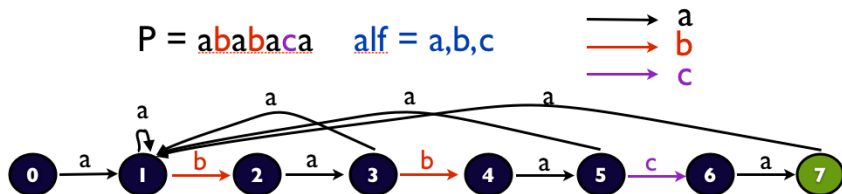
5 - Construcción de autó

→ a
→ b
→ c

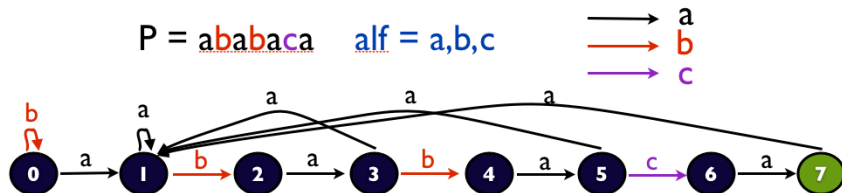




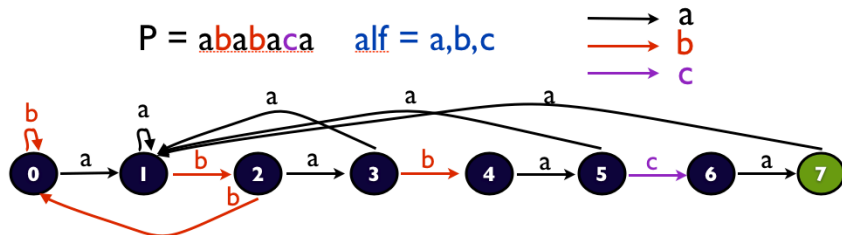
Automatas finitos - Construcción de autómeta

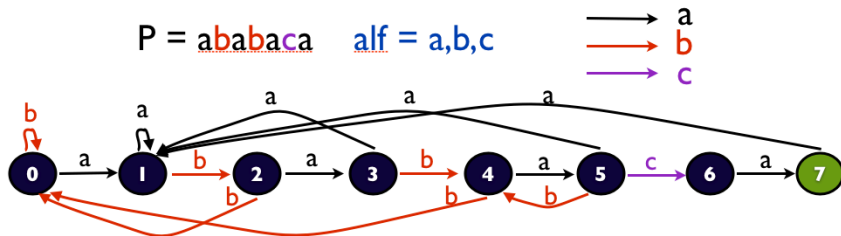


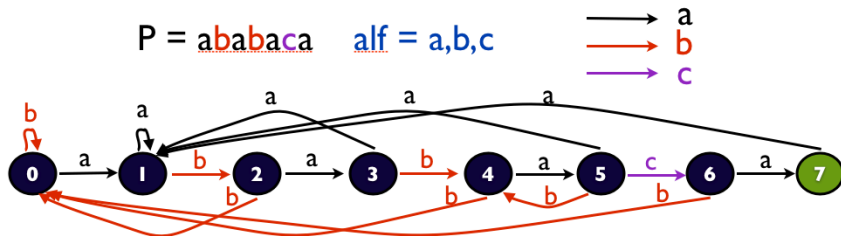
5 - Construcción de autó



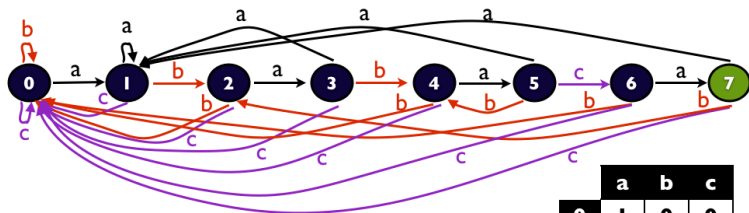
Automatas finitos - Construcción de autómata







Automatas finitos - Construcción de autómeta



$P = \underline{a} \underline{b} \underline{a} \underline{b} \underline{a} \underline{c} \underline{a}$ $\underline{a} \underline{f} = a, b, c$

—→ a
 —→ b
 —→ c

	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	5	0	0
5	1	4	6
6	7	0	0
7	1	2	0

Knuth-Morris-Pratt - Construcción de los prefijos

P = ababababca

P = ababababca

Sufijo **ab** es prefijo de longitud 2

PI =

1	2	3	4	5	6	7	8	9	10
0	0	1	2	3	4	5	6	0	1

Knuth-Morris-Pratt - Construcción del arreglo

q	k	P
2	0	ababababca
3	0	ababababca
4	1	ababababca
5	2	ababababca
6	3	ababababca
7	4	ababababca
8	5	ababababca
9	6	ababababca
9	4	ababababca
9	2	ababababca
9	0	ababababca
10	0	ababababca
	1	

PI

1	2	3	4	5	6	7	8	9	10
0	0								
		1							
			2						
				3					
					4				
						5			
							6		
								0	
									1
0	0	1	2	3	4	5	6	0	1

Para todo $q \leftarrow 2$ a m

- 1) loop $k > 0$ and $P[k+1] \neq P[q]$? $k = PI[k]$
- 2) $P[k+1] = P[q]$? $k = k+1$
- 3) $PI[q] = k$

[illegible]

```
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
i = 9
i = 10
i = 11
i = 12
```

Emparejamiento a $i - |P| = 2$

```

1) loop q>0 & P[q+1] <> T[i]? q = PI[q]
2) P[q+1]=T[i]? q = q+1
3) q = |P|? emp. en i-|P|, q = PI[q]

```

1	2	3	4	5	6	7	8	9	10
0	0	1	2	3	4	5	6	0	1

ión

UMSS

er Vladimir Levenshtein, en 196

O de costo $O(m * n)$

tud de una de las palabras

ud de la otra.

- ▶ Planteado por Vladimir Levenshtein, en 1965.
- ▶ Algoritmo PD de costo $O(m * n)$
- ▶ m es la longitud de una de las palabras
- ▶ n es la longitud de la otra.

Distancia de Edición

El algoritmo es ir verificando a medida que se leen las cadenas si es necesario hacer una eliminación, inserción, sustitución o nada.

El razonamiento es simple:

Si se quiere definir la distancia entre $a[1, 2, \dots, i]$ y $b[1, 2, \dots, j]$ se puede dar por hecho que se sabe la distancia de emparejar:

$a[1, 2, \dots, i-1]$	y	$b[1, 2, \dots, j]$
$a[1, 2, \dots, i-1]$	y	$b[1, 2, \dots, j-1]$
$a[1, 2, \dots, i]$	y	$b[1, 2, \dots, j-1]$

Distancia de Edición

Por ejemplo: si se quiere definir la distancia entre “aurora” y “neurona”, se da por hecho que ya se sabe la distancia entre

auror	y	neurona
auror	y	neuron
aurora	y	neuron

Distancia de Edición

El algoritmo mas sencillo considera que cualquier operación de edición se cuenta como 1.

En el caso del ejemplo:

-aurora

neurona

^^ ^

DC C ---> Delete, Change

La distancia de edición es 3

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
		i	n e u r o n a							
0		0								
1	a									
2	u									
3	r									
4	o									
5	r									
6	a									

cadena vacía
cadena vacía
diferencia es 0

Distancia de Edición - Básico

i \ j	0	1	2	3	4	5	6	7
	n e u r o n a							
0	0	1						
1	a							
2	u							
3	r							
4	o							
5	r							
6	a							

cadena vacía
cadena "n"
diferencia es 1

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
			neurona							
i	0	0	1	2						
1	a									
2	u									
3	r									
4	o									
5	r									
6	a									

cadena vacía
cadena "ne"
diferencia es 2

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
		i	n e u r o n a							
0	a	0	0	1	2	3	4	5	6	7
1	a									
2	u									
3	r									
4	o									
5	r									
6	a									

cadena vacía
cadena x
diferencia es $|x|$

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
			n e u r o n a							
i			0	1	2	3	4	5	6	7
0			0	1	2	3	4	5	6	7
1	a		1							
2	u									
3	r									
4	o									
5	r									
6	a									

cadena "a"
cadena vacía
diferencia es 1

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
		i	n e u r o n a							
0			0	1	2	3	4	5	6	7
1	a		1							
2	u		2							
3	r									
4	o									
5	r									
6	a									

cadena "au"
cadena vacía
diferencia es 2

Distancia de Edición - Básico

		j								
		0	1	2	3	4	5	6	7	
i		n e u r o n a								
0		0	1	2	3	4	5	6	7	
1	a	1								
2	u	2								
3	r	3								
4	o	4								
5	r	5								
6	a	6								



cadena x
cadena vacía
diferencia es $|x|$

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
i			n e u r o n a							
0			0	1	2	3	4	5	6	7
1	a	1								
2	u	2								
3	r	3								
4	o	4								
5	r	5								
6	a	6								

cadena "a"
cadena "n"
existe diferencia
entonces operación
∴ 1 +
* dist("", "n")
* dist("", "")
* dist("a", "")

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
i			n e u r o n a							
0			0	1	2	3	4	5	6	7
1	a	1	1							
2	u	2								
3	r	3								
4	o	4								
5	r	5								
6	a	6								

cadena "a"
cadena "n"
existe diferencia
entonces operación
:. 1 +
* dist("", "n") = 1
* dist("", "") = 0
* dist("a", "") = 1

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
i			n e u r o n a							
0			0	1	2	3	4	5	6	7
1	a	1	1	2						
2	u	2								
3	r	3								
4	o	4								
5	r	5								
6	a	6								

cadena "a"
cadena "ne"
existe diferencia
entonces operación

∴ 1 +

* $\text{dist}("", "ne") = 2$

* $\text{dist}("", "n") = 1$

* $\text{dist}("a", "n") = 1$

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
i			n e u r o n a							
0		0	1	2	3	4	5	6	7	
1	a	1	1	2	3	4	5	6	6	
2	u	2								
3	r	3								
4	o	4								
5	r	5								
6	a	6								

cadena "a"
cadena "neurona"
no existe diferencia
entonces operación

∴ 0 +

* $\text{dist}("", "neurona") = 7$

* $\text{dist}("", "neuron") = 6$

* $\text{dist}("a", "neuron") = 6$

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
		i	n e u r o n a							
0		0	0	1	2	3	4	5	6	7
1	a	1	1	2	3	4	5	6	6	
2	u	2	2							
3	r	3								
4	o	4								
5	r	5								
6	a	6								

cadena "au"
cadena "n"
existe diferencia
entonces operación

∴ 1 +

* $\text{dist}("a", "n") = 1$

* $\text{dist}("a", "") = 1$

* $\text{dist}("au", "") = 2$

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
		i	n e u r o n a							
0		0	0	1	2	3	4	5	6	7
1	a	1	1	2	3	4	5	6	6	
2	u	2	2	2	2					
3	r	3								
4	o	4								
5	r	5								
6	a	6								

cadena "au"
cadena "neu"
no existe diferencia
entonces operación

∴ 0 +
* $\text{dist}(\text{"a"}, \text{"neu"}) = 3$
* $\text{dist}(\text{"a"}, \text{"ne"}) = 2$
* $\text{dist}(\text{"au"}, \text{"ne"}) = 2$

Distancia de Edición - Básico

		j	0	1	2	3	4	5	6	7
i			n e u r o n a							
	0		0	1	2	3	4	5	6	7
	1	a	1	1	2	3	4	5	6	6
	2	u	2	2	2	2	3	4	5	6
	3	r	3	3	3	3	2	3	4	5
	4	o	4	4	4	4	3	2	3	4
	5	r	5	5	5	5	3	3	3	4
	6	a	6	6	6	6	4	4	4	3

cadena "aurora"
cadena "neurona"
no existe diferencia
entonces operación
∴ 0 +
* dist("auror","neurona") = 4
* dist("auror","neuron") = 3
* dist("aurora","neuron") = 4

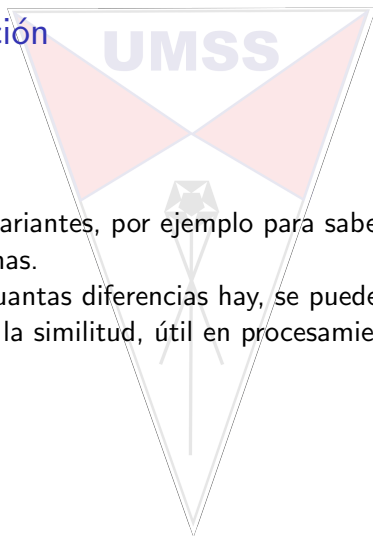
Distancia de Edición - Básico

Complejidad de algoritmo $O(n*m)$, solución con PD

		j	0	1	2	3	4	5	6	7
			n e u r o n a							
i			0	1	2	3	4	5	6	7
0			0	1	2	3	4	5	6	7
1	a		1	1	2	3	4	5	6	6
2	u		2	2	2	2	3	4	5	6
3	r		3	3	3	3	2	3	4	5
4	o		4	4	4	4	3	2	3	4
5	r		5	5	5	5	3	3	3	4
6	a		6	6	6	6	4	4	4	3

```
cad1[1..i]
cad2[1..j]
diff(cad1i, cad2j)
= 0 si cad1i = cad2j
= 1 si cad1i ≠ cad2j
dist[i,j] = diff(cad1i, cad2j) +
min(
    dist[i-1,j] ,
    dist[i-1,j-1] ,
    dist[i,j-1]
)
```

Distancia de Edición



Es utilizado con variantes, por ejemplo para saber cuanto se parecen dos cadenas.

En vez de decir cuantas diferencias hay, se puede encontrar un score que indique la similitud, útil en procesamiento de ADN

Distancia de Edición - Algoritmo

Hay varios . . se tomará la de *Neddleman Wunsch's*. Este algoritmo considera las posibilidades de emparejamiento letra a letra (A_i, B_i), y asigna un “score” dependiendo de como son los caracteres: iguales o distintos.

- ▶ si $A_i = B_i \implies$ score es $+2$
- ▶ si $A_i \neq B_i \implies$ score es -1 ; que seria equivalente a:
 - ▶ cambiar A_i por B_i ,
 - ▶ insertar un espacio en A_i en la cadena A
 - ▶ eliminar A_i de la cadena A

Distancia de Edición - Algoritmo general

Realizar una matriz v que permita comparar las cadenas A y B . A en las filas y B en las columnas.

$sc(a, b) = a == b ? 2 : -1;$

Base

$v(0, 0) = 0$

$v(i, 0) = i * sc(A[i], _)$ // eliminar subcadena $A[1..i]$

$v(0, j) = j * sc(_, B[j])$ // insertar espacios $B[1..j]$

Distancia de Edición - Algoritmo general

Emparejar

$\forall i, j / i, j > 0$

$opc1 = v(i-1, j-1) + sc(A[i], B[j])$

$opc2 = v(i-1, j) + sc(A[i], _)$

$opc3 = v(i, j-1) + sc(_, B[j])$

$v(i, j) = \max(opc1, opc2, opc3)$

ión - Ejecución

$$\begin{aligned} v(0,0) &= 0 \\ v(i,0) &= -1*i \\ v(0,j) &= -1*j \end{aligned}$$

ión - Ejecución

ión - Ejecución

v(10, 8) esta la resp